Hyperparameter Tuning of NSGA-II for Model-Based Concrete Traffic Scene Generation

Catriona B.K. McIntosh
McGill University
Montreal, Canada
catriona.mcintosh@mail.mcgill.ca

Aren A. Babikian
McGill University
Montreal, Canada
aren.babikian@mail.mcgill.ca

***Abstract*—Meta-heuristic search-based approaches such as multi-objective evolutionary algorithms (MOEAs) have commonly been used to address modelling problems, such as in the domain of autonomous vehicle testing, where deterministic algorithms fail to provide exact solutions in reasonable time. A drawback of using MOEAs is that performance is highly reliant on hyperparameter values which may only be optimized through systematic evaluation, often not included in existing research.**

In this experience paper, we systematically evaluate the influence of hyperparameter tuning of the NSGA-II MOEA applied to a concrete traffic scene generation challenge involving geometric attribute constraints that form complex constraint structures. Specifically, we address the *population size* and *number of offspring* hyperparameters, and we evaluate success rate and runtime of the proposed approach. As a long-term research objective, we plan to build upon the reported results to analysis their generalizability over various domains and MOEA configurations.

***Index Terms*—hyperparameter tuning, multi-objective optimization, concrete traffic scene generation**

I. Introduction

Meta-heuristic search-based approaches such as multi-objective evolutionary algorithms (MOEAs) have commonly been used to address complex modelling problems where deterministic algorithms fail to provide exact solutions in reasonable time [1], [2]. This particularly applies to modelling problems that require the handling of numeric attribute constraints which are often not only mathematically complex (e.g. non-linear constraints), but also form complex constraint structures (e.g. cyclic constraints). As such, despite the existence of performant deterministic numeric solvers such as dReal [3], existing research relies on MOEAs to handle such problems.

A characteristic of MOEAs is that they are known to require tuning of their hyperparameters to optimize their performance. Additionally, different optimization-based approaches, even within the same domain, propose different MOEA configurations, which suggests that optimal hyperparameter values may not be transferable between approaches. Therefore, systematic evaluation of a given approach is required to determine its optimal hyperparameter values. Such systematic evaluation is often missing from existing MOEA-based approaches: authors only report exact hyperparameter values being used (commonly based on *general-purpose* guidelines for MOEAs).

This paper is partially supported by the NSERC PGSD3-546810-2020 scholarship.

In this experience paper, we systematically evaluate the influence of hyperparameter tuning on the NSGA-II [4] MOEA applied to a concrete traffic scene generation challenge within the domain of autonomous vehicle (AV) testing. The addressed challenge defines a object placement problem with complex geometric attribute constraints where a single result is derived. We report results for the tuning of *population size* and *number of offspring* that will contribute to a longer-term research objective involving a wider scope (e.g. hyperparameter tuning analysis over multiple domains and MOEA configurations).

II. Related work

Modelling challenges as optimization problems: In order to use MOEAs such as NSGA-II [4] to address a modelling challenge, it must be represented as a *constrained* Multi-Objective Optimization Problem (MOP), defined by a set of objectives (which need to be optimized) and a set of constraints (which are required to be satisfied by a solution) [5]. Specific objectives and constraints are determined according to the particular requirements of the addressed challenge.

MOEAs may be domination-based (i.e. solutions are ranked via non-dominated sorting) or decomposition-based (i.e. the problem is split into multiple single-objective optimization subproblems that are solved collaboratively). Furthermore, different MOEAs handle constraints and objectives differently: constraints may be handled independently from objectives (through, e.g. constraint dominance principle (CDP) [4] or stochastic ranking (SR) [6]), or they may be converted into objectives and handled as such [7].

In the domain of AV testing, existing research most commonly extends general-purpose domination-based MOEAs. For instance, existing work extends NSGA-II with surrogate models [1], decision-tree learning [2] or novelty search [8], while others simply adapt the default constraints and objectives of NSGA-II [9]. In all cases, the performance of evolutionary algorithms is significantly impacted by the tuning of hyperparameters such as *population size* and *number of offspring* [10], [11], which guide the exploration-exploitation tradeoff involved in MOEAs. Optimal hyperparameter values depend on the particular MOEA being used as well as on the encoding of constraints and objectives, and are hard to determine statically (without systematic evaluation).

Systematic analysis of hyperparameter tuning: Few existing case studies provide systematic analysis of hyperparameter tuning in MOEAs. For instance, in the domain of cryptography, a genetic algorithm (GA) is used in AES to strengthen the key generated by AES-128 [12]. The algorithm used uses *roulette wheel selection*, *inversion mutation*, and *uniform crossover*, with a crossover rate of 0.8 and a mutation rate of 0.3. *Population sizes* between 10 and 1000 were tested, with 200 yielding optimal encryption and decryption times.

Population sizes between 2 and 4096 are evaluated on a node placement problem in Wireless Mesh Networks [13]. Performance quality is measured based on the size of the giant component and the number of covered users parameters. The case study uses a GA with *linear ranking selection*, while the mutation and crossover operations are based on domain-specific parameters related to the grid-based encoding of individuals. Mutation and crossover rates are 0.2 and 0.8, respectively. Reported results show better performance with increasing *population size*. However, with the increase of the *population size*, the GA requires more computational time.

In this paper, we perform systematic analysis of hyperparameter tuning on a *domination-based multi-objective evolutionary algorithm*, namely NSGA-II [4], applied to an optimization problem. The derivation of the problem is detailed in Section III, alongside the relevant genetic operators, fitness functions and termination criteria.

III. Overview

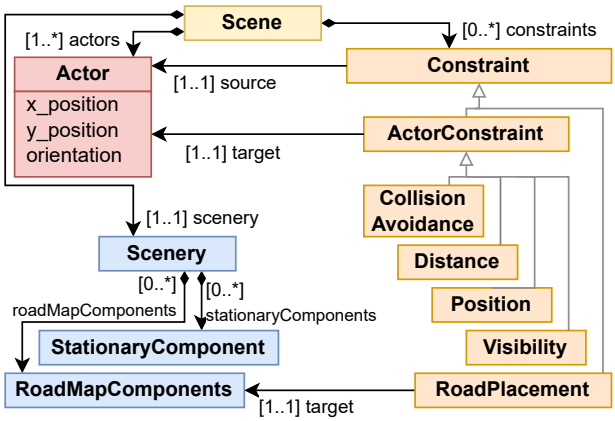


Fig. 1: A domain model for traffic scenes

A. Concrete scene generation as a modelling challenge

We publish the empirical performance evaluation of hyperparameter tuning over an existing concrete traffic scene generation challenge proposed in our related work that is currently under review for publication [14]. Figure 1 presents the domain model used for concrete scene generation, where a scene (defined as a static snapshot of the environment [15]) is composed of:

- the *Scenery*, which contains (1) *road map components* such as roads and junctions, and (2) *stationary components* such as traffic lights,
- at least one *Actor* (e.g. vehicles), which have numeric *position* coordinates and an *orientation* angle, and
- (Geometric) *Constraints* over actors. In this paper, we address five types of constraints:
  - *Collision avoidance* constraints assert that two actors are not overlapping,
  - *Distance* and *position* constraints use qualitative abstractions to define the distance (e.g. close, far) and positional relationship (e.g. behind, to the left of) between two actors, respectively,
  - *Visibility* constraints assert that the target actor is visible for the source actor, and
  - *Road Placement* constraints assert that the source actor is placed on the target road map component.

The concrete scene generation challenge addressed in this paper takes as input an (abstract) scene, as described above, where the value of (concrete) numeric attributes of actors are unknown. We aim to assign exact numeric attribute values for actors (i.e. to lay out the actors over the road map) such that all constraints are satisfied. We handle binary geometric (trigonometric and quadratic) attribute constraints over complex road maps. Furthermore, we handle complex constraint structures (constraints may form an arbitrary directed graph).

B. Concrete scene generation as a MOP

To address the concrete scene generation challenge described above, we represent it as a multi-objective minimization problem. For each actor of the input scene, we associate two numeric variables (which represent the position coordinates) bounded by the road map size in the derived MOP. The orientation of each actor is determined according to its position on the road map. For simplicity, we consider actors with fixed dimensions.

The objectives of the MOP are based on fitness function calculations over constraint instances according to their type. For *distance* constraints, fitness is determined by the difference between the measured distance and the target distance range. As such, the fitness function is minimized when the objects are within the desired distance range from each other. Similarly, for (1) *road placement* and (2) *visibility* constraints, fitness is determined by measuring the distance between the source actor and (1) the target road map component, or (2) the field of view of the target actor, respectively. For *position* constraints, fitness is based on the difference in angle between the target actor and the desired position relative to the source actor. Finally, for *collision avoidance* constraints, fitness is minimized if the involved actors are overlapping. Otherwise, it is constant.

To define the objectives of the MOP, we aggregate fitness functions (i.e. we return the sum of all fitness function values) according to the type of constraint they are associated with. For instance, all fitness function values for visibility constraints in a scene are summed together to define an objective. As such, our MOP will contain 5 objectives, for the 5 constraint types we handle. Note that all constraints are integrated into the MOP as objectives.

We also apply a power function to the objective value, according to the importance we associate to each constraint category. Specifically, to increase the realisticness of the derived solutions, we apply a cube operator to the objectives associated to collision avoidance and road placement constraints, while we apply a square operator to all other objectives.

C. MOP configuration parameters

In this paper, we perform systematic analysis of the *population size* and *number of offspring* hyperparameters of NSGA-II applied to the concrete scene generation challenge. An NSGA-II run is terminated (1) if a (single) solution that satisfies all constraints (i.e. all objective values are minimized) is found, or (2) if the run times out. This is a particularity of our approach: we leverage the fact that all objectives have a reachable lower bound (which represents the optimal value) and we do not consider non-optimal solutions.

For genetic operators, we use default values as defined in the PyMoo [16] implementation of the algorithm. Namely, we use (1) *Simulated Binary Crossover* [17] with probability 0.9 and distribution index 15, (2) *Polynomial Mutation* [17] with probability $\frac{1}{2 \times n_{actors}}$ nd distribution index 20, (3) *Fitness Domination and Crowding Distance Binary Tournament* as the selection operator, and (4) *Non-Dominated Sorting Rank and Crowding Distance* as the Survival operator.

IV. Evaluation

We conduct measurements to address the following research questions:

RQ1: How does hyperparameter tuning on *population size* and *number of offspring* impact the *success rate* of concrete traffic scene generation?

RQ2: How does hyperparameter tuning on *population size* and *number of offspring* impact the *runtime* of concrete traffic scene generation?

A. General setup:

To evaluate the performance of various hyperparameter settings, we run measurements on the *Town02* map included in the CARLA simulator framework [18]. We run measurements over abstract scenes generated using randomized techniques that ensure the existence of a feasible solution. This is done by randomly placing n actors within a distance r from each other on a map while enforcing road placement and collision avoidance constraints. We then derive a full set of constraints from the exact placement of actors through qualitative abstractions. Measurements are conducted in a Python environment on an enterprise server[1], and the garbage collector is called explicitly between runs.

Population size and *number of offspring* were initially tested at values 5-30 in increments of 5, considering that their default values in the PyMoo [16] implementation of NSGA-II is 20 and 10, respectively. Given our initial results favoured lowered *population sizes*, we wanted to explore the lower limit on *population size* and see if this observation continued with larger *population sizes*. Thus, we expanded the search to 1-4, 40, 50, 60. For consistency, we also explored these values for *number of offspring*. While experimenting with one hyperparameter, all others were fixed at default implementation values. However, given the results from our initial *population size* experiment, *population size* was set to 5 when tuning *number of offspring*.

In total, measurements were run on 30 different randomly generated scenes equally divided into 2, 3, and 4 actor scene specifications. Larger scene sizes are excluded to ensure higher success rates. Furthermore, scene concretization was evaluated ten times on each scene specification for each hyperparameter setting. Time out was set to 10 minutes for each run.

[1] 12×2.2 GHz CPU, 64 GiB RAM, CentOS 7, Java 1.8, 12 GiB Heap

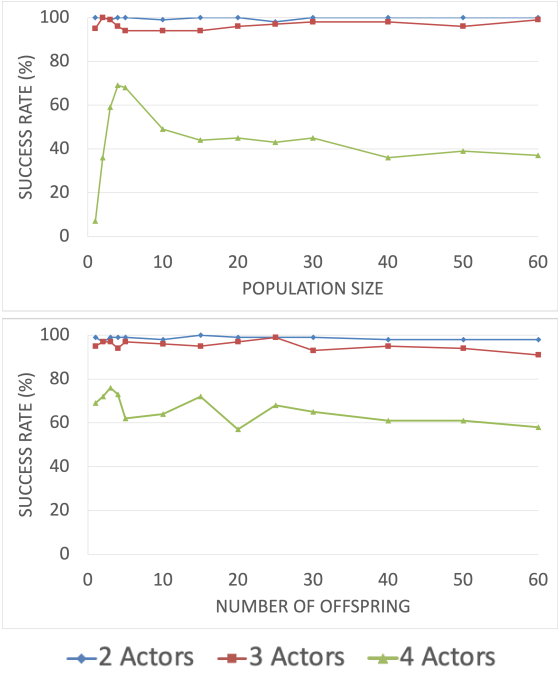


Fig. 2: Success rate of concrete scene generation wrt. *population size* and *number of offspring*.

B. RQ1: Success Rate

We compare the success rate of concrete scene generation wrt. the hyperparameters *population size* and *number of offspring* in Figure 2. We consider a run to be successful if the solution generated by concrete scene generation within the time limit satisfies all constraints.

For 2 and 3 actors, the concrete scene generation approach achieves a high success rate of at least 91% irrespective of the hyperparameters. Conversely, with 4 actors, scene concretization is more difficult and we see more variation in success for different hyperparameter configurations. Given this, we focus on the performance of these hyperparemeters in 4 actor scenes. For statistical significance we perform the *Fisher exact test* [19] to determine p-value and measure the *odds ratio* [20] for effect size as suggested in existing guidelines [21]. This is done between each *population size* and also between each *number of offspring* value (pairwise).

The worst performing *population size*, 2, resulted in a success rate of only 7%, while the best performer, 4, had a 69% success rate. In fact, 2 is significantly slower than all other *population sizes* according to *p-values* ($p < 0.05$). Both 4 and 5 are significantly faster than all but one other configuration (with effect sizes greater than 2.2). The influence of *number of offspring* is less notable. We find the highest success rate to be 72% with 3 offspring, and the lowest success rate 57% with 20 offspring. Moreover, the *number of offspring* value with the highest success rate is only faster than five of the eleven other configurations according to statistical significance.

Figure 2 shows that success rate for 4 actors peaks with approximately 70% success rate near *population size* 4 and 5, and stagnates (around 40%) as *population size* grows. A dominant peak is not present for success rate wrt. *number of offspring*, however, we do notice a slight decrease as *number of offspring* increases past 25.

RQ1: *Population sizes 4 and 5 provide highest success rates, while the influence of the number of offspring on success rate is less notable.*

C. RQ2: Runtime

We compare the runtime wrt. *population sizes* and *number of offspring* values as a colour coded boxplots in Figure 3a and Figure 3b respectively. We do not include the runtimes from unsuccessful attempts as prescribed in [21]. For statistical significance of runtime measurements, we perform the *Mann-Whitney U-test* [22] to determine *p-value* and measure the Vargha and Delaney's $\hat{A}_{12}$ [23] for effect size.

Given that 2 actor scenes finish in under $8.1s$ for all configurations we exclude them from our analysis. For 3 and 4 actor scenes, *population size* 5 with the fastest median runtime is faster with statistical significance ($p < 0.05$) than at least nine of the eleven other configurations. The slowest median runtimes are slower than at least seven other configurations with statistical significance. This describes the high and low peaks shown in Figure 3a. Effect sizes are less than 1.0.

Analyzing the runtime results for *number of offspring*, the quickest median runtime for 3 actor scenes is faster than nine other settings, while the slowest runtime is slower than nine configurations. For 4 actor scenes, the fastest configuration is faster than eight other configurations, however the slowest is only slower than 2 other values. 4 if the fastest *offspring* setting for both scene sizes. These claims are supported by statistical significance (effect sizes < 1.0). This indicates that for the more difficult 4 actor scenes, one configuration produces a fast runtime relative to the other settings, however most *number of offspring* values produce similar runtimes.

As *population size* changes across all scene sizes, the runtime decreases with increasing *population size* until a certain value, after which all larger *population sizes* have greater runtimes. For 3 and 4 actors, this trend is centered around *population size* 5. We notice that for 4 actors, after *population size* 5, runtimes slope down to *population size* 30 and then begin to rapidly grow again. We observe that *number of offspring* 1, 2, 3, 4 are consistently among the best performers. However, there is no obvious trend and all other configurations of *number of offspring* achieve variable runtime performance for different scene sizes.

RQ2: *Tuning of population size demonstrates a trend with minimum runtime at 5. We also notice that smaller offspring values are generally faster. Furthermore, results show that runtime is more sensitive to population size.*

V. Discussion

Based on our results, we conclude that 5 is the optimal value for *population size* in the concrete scene generation approach using NSGA-II as the underlying search algorithm. Considering that success rates are consistently high for 2 and 3 actors, we rely on 4 actor scenes to derive the optimal setting. Figure 2 shows that the success rate is dominated by *population sizes* 3, 4 and 5 (with statistical significance for *population sizes* 4 and 5). Furthermore, *population size* 5 outperforms *population size* 4 (and at least 9 of the 11 other configurations) in terms of runtime, with statistical significance (despite low effect sizes), for both 3 and 4 actor scenes.

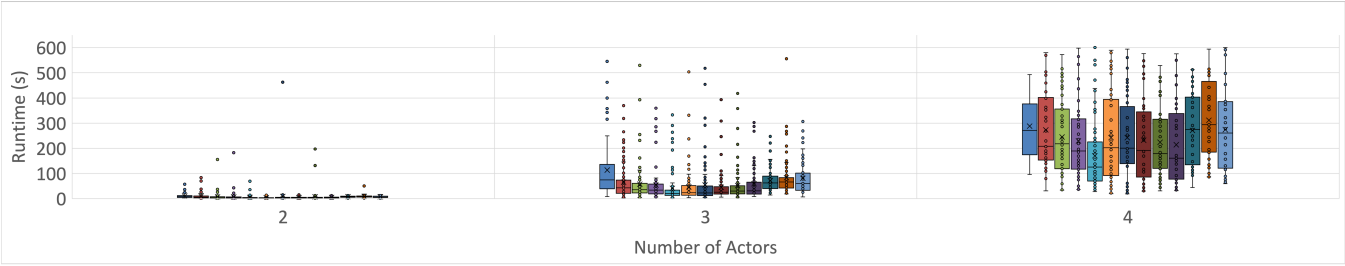
For *number of offspring*, our results for both success rate and runtime show that 3 and 4 are consistently among the best performers. Nevertheless, their statistical significance is not as strong as for *population size* as discussed in Section IV, which does not allow us to make exact suggestions pertaining to optimal hyperparameter values. Moreover, we do notice a general trend that performance generally decreases wrt. both success rate and runtime as the *number of offspring* increases.

Although we provide concrete numeric suggestions for optimal hyperparameter values for the specific problem and MOEA configuration addressed in this paper, existing research provides no evidence that these results may be directly transferable to other domains or to other MOEA configurations. Neverthless, we plan to build upon the results reported in this paper to analyze cross-domain and cross-configuration transferability of hyperparameter tuning for MOEAs as a long-term research objective.

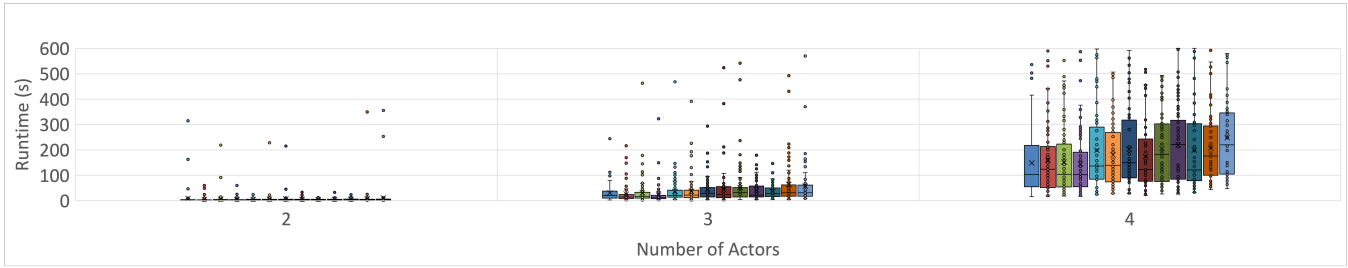
VI. Conclusions

We conducted hyperparameter tuning on *population size* and on *number of offspring* to optimize the performance of NSGA-II applied to a concrete scene generation problem for placing vehicles over an arbitrary road map such that they satisfy complex geometric constraints given as input.

Measurements were carried out comparing the performance of thirteen configurations of *population size* and of *number of offspring*. We evaluate the success rate and runtime of concrete scene generation. Our results, supported by statistical significance measurments, show that 5 is the optimal setting of *population size* for concrete scene generation. *Number of offspring* performed best with values of 3 or 4, however the performance benefits over other settings was limited. Overall, *population size* is more influential than *number of offspring* on the performance of concrete scene generation. In future work, we will explore the transferability of our results to other domains and configurations.



(a) R2: *Population size*



(b) RQ2: *Number of offspring*

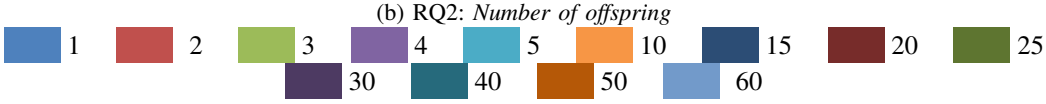


Fig. 3: Successful runtimes of scene concretization wrt. hyperparameter tuning on *population size* and *number of offspring*. The colours represent the setting of *population size* and *number of offspring*, respectively.

References

[1] R. B. Abdessalem, S. Nejati, L. C. Briand, and T. Stifter, "Testing advanced driver assistance systems using multi-objective search and neural networks," *ASE 2016 - Proceedings of the 31st IEEE/ACM Int'l Conf. on Automated Software Engineering*, pp. 63–74, 2016.

[2] ——, "Testing vision-based control systems using learnable evolutionary algorithms," in *Proceedings - Int'l Conf. on Software Engineering*, vol. 11, no. 18. IEEE Computer Society, 2018, pp. 1016–1026.

[3] S. Gao, J. Avigad, and E. M. Clarke, "δ-complete decision procedures for satisfiability over the reals," in *Lecture Notes in Computer Science*, vol. 7364 LNAI, 2012, pp. 286–300.

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[5] Z. Fan, F. Yi, W. Li, J. Lu, X. Cai, and C. Wei, "A comparative study of constrained multi-objective evolutionary algorithms on constrained multi-objective optimization problems," *2017 IEEE Congress on Evolutionary Computation, CEC 2017 - Proceedings*, pp. 209–216, 2017.

[6] T. P. Runarsson and X. Yao, "Stochastic ranking for constrained evolutionary optimization," *IEEE Transactions on Evolutionary Computation*, vol. 4, pp. 284–294, 2000.

[7] Z. Cai and Y. Wang, "A multiobjective optimization-based evolutionary algorithm for constrained optimization," *IEEE Transactions on Evolutionary Computation*, vol. 10, pp. 658–675, 2006.

[8] V. Riccio and P. Tonella, "Model-based exploration of the frontier of behaviours for deep learning system testing," *Proceedings of the 28th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of SE*, pp. 876–888, 2020.

[9] S. Wu, H. Wang, W. Yu, K. Yang, D. Cao, and F. Wang, "A new sotif scenario hierarchy and its critical test case generation based on potential risk assessment," *Proceedings 2021 IEEE 1st Int'l Conf. on Digital Twins and Parallel Intelligence, DTPI 2021*, pp. 399–409, 2021.

[10] M. Odetayo, "Empirical study of the interdependencies of genetic algorithm parameters," in *EUROMICRO 97. Proceedings of the 23rd EUROMICRO Conference*, 1997, pp. 639–643.

[11] K. D. Jong, "Parameter setting in eas: a 30 year perspective," in *SCI*. Springer, 2007, vol. 54, pp. 1–18.

[12] K. Kalaiselvi and A. Kumar, "An empirical study on effect of variations in the population size and generations of genetic algorithms in cryptography," in *2017 IEEE International Conference on Current Trends in Advanced Computing (ICCTAC)*, 2017, pp. 1–5.

[13] T. Oda, A. Barolli, E. Spaho, L. Barolli, F. Xhafa, and M. Takizawa, "Performance evaluation of wmn-ga system for low densities of clients and different settings of population size," in *2012 Sixth International Conference on Complex, Intelligent, and Software Intensive Systems*, 2012, pp. 314–319.

[14] A. A. Babikian, O. Semeráth, and D. Varró, "Concretization of abstract traffic scene specifications using metaheuristic search," *IEEE Transactions on Software Engineering*, [Submitted for review].

[15] S. Ulbrich, T. Menzel, A. Reschka, F. Schuldt, and M. Maurer, "Defining and substantiating the terms scene, situation, and scenario for automated driving," in *IEEE Conference on Intelligent Transportation Systems, Proceedings, ITSC*, vol. 2015-Octob. IEEE Inc., 2015, pp. 982–988.

[16] J. Blank and K. Deb, "Pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[17] K. Deb, K. Sindhya, and T. Okabe, "Self-adaptive simulated binary crossover for real-parameter optimization," *Proceedings of GECCO 2007: Genetic and Evolutionary Computation Conference*, pp. 1187–1194, 2007.

[18] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, 2017, pp. 1–16.

[19] R. A. Fisher, "Statistical methods for research workers," *Breakthroughs in statistics*, pp. 66–70, 1992.

[20] C. K. Haddock, D. Rindskopf, and W. R. Shadish, "Using odds ratios as effect sizes for meta-analysis of dichotomous data: A primer on methods and issues," *Psychological Methods*, vol. 3, no. 3, pp. 339–353, 1998.

[21] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," *Proceedings - Int'l Conf. on Software Engineering*, pp. 1–10, 2011.

[22] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[23] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong:," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.